

Lógica y Representación 1

Funciones - Introducción

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

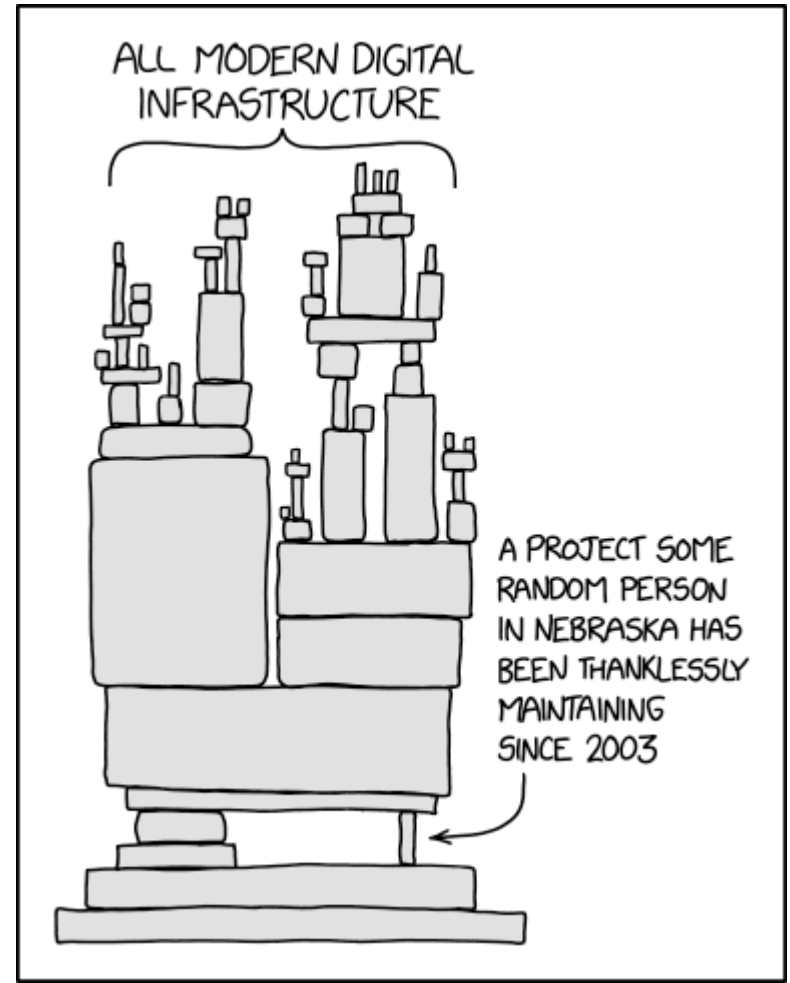


Outputs



Introducción a las funciones

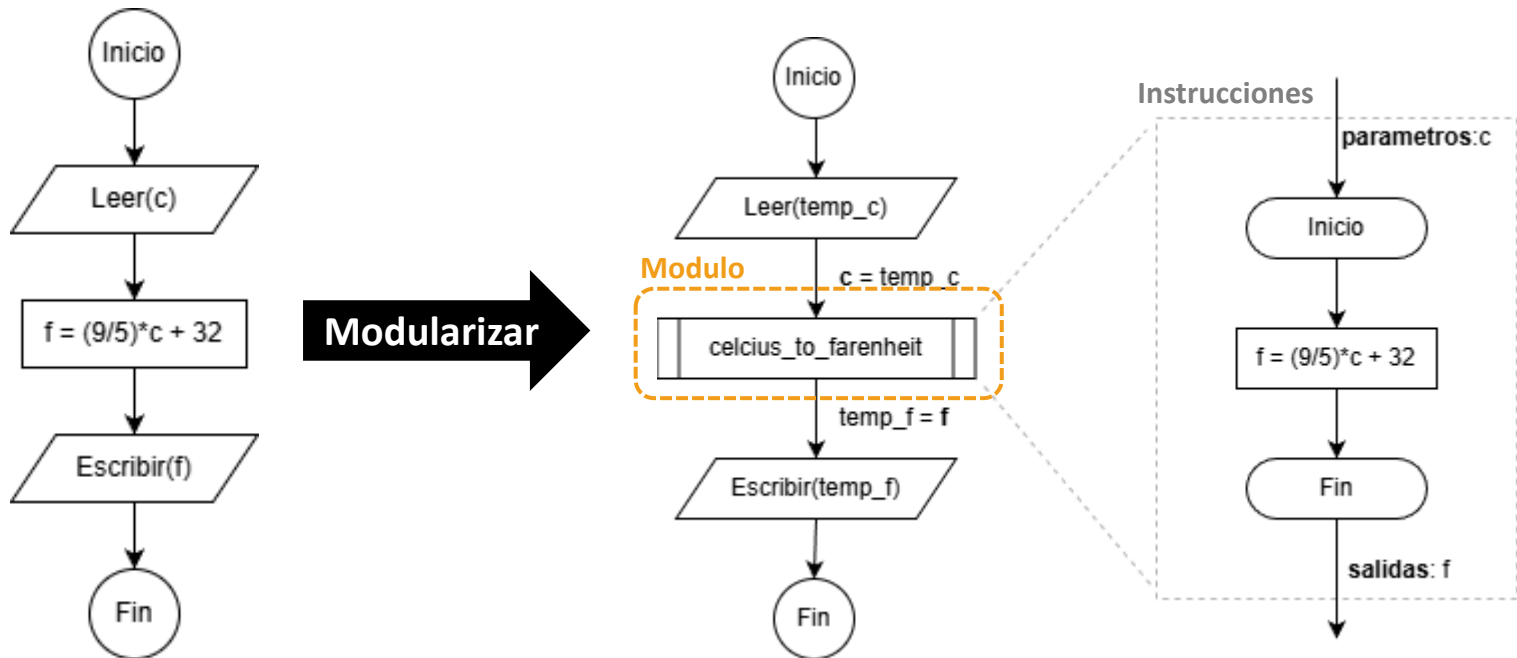
```
int getRandomNumber()  
{  
    return 4; // chosen by fair dice roll.  
              // guaranteed to be random.  
}
```



Función

Función:

- Porción de código que realiza una tarea específica.
- También se conoce como: subrutina, procedimiento, función o método.



```
c = float(input("Ingrese los °C: "))
f = (9.0/5)*c + 32
print(c,"°C =",f, "°F")
```

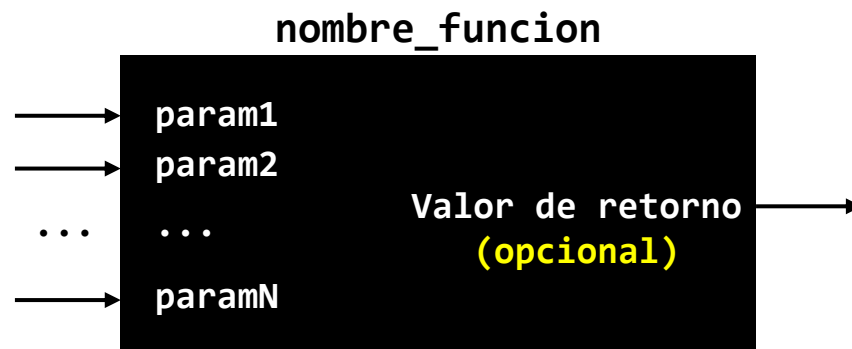
[Link simulación](#)

Funciones en Python - Definición

- **Sintaxis:** Una función se define mediante la palabra clave **def** seguido del nombre del nombre y los parámetros entre paréntesis:

```
def nombre_funcion(param1, param2 ,... , paramN):  
    cuerpo de la funcion
```

- **Donde:**
 - **Nombre de la función:** Identifica la función; con este se llama (invoca).
 - **Parámetros:** Lista de entradas de la función (puede estar vacía).
 - **Cuerpo de la función:** Conjunto de instrucciones que realizan la tarea (van indentadas).
- **Importante:** Definir una función no la ejecuta: solo se ejecuta cuando se la llama.
- **Diagrama de caja negra:** Resume la interfaz de la función (entradas/salidas) sin entrar en los detalles internos.



Funciones en Python - Invocación

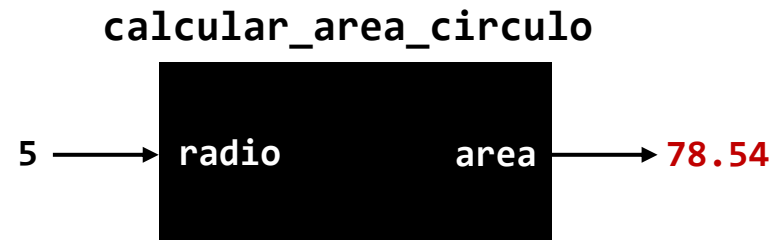
- **Invocar (llamar) una función:** escribir su nombre con los argumentos entre paréntesis

```
[variable =] nombre_funcion(arg1, arg2 ,... , argN)
```

- **Donde:**
 - **Argumentos:** los valores que se envían, y se asocian a los parámetros **por posición**. Pueden ser números, variables o expresiones.
 - Si la función **devuelve** un valor, se guarda en una variable o se usa directamente. Si no devuelve nada, se la llama sola.
- **Ejemplo:** Empleando la función `calcular_area_circulo()`, calcular el área de un círculo de radio 5 y llevar el resultado a una variable llamada área.

```
area = calcular_area_circulo(5)
```

[Link simulación](#)



Probar una función

- **Prueba unitaria:** verificar una función de forma aislada, con varios casos de uso.
- Cada **caso de prueba** compara el resultado **esperado** con el **obtenido**.
- **Integración:** proceso de combinar las funciones ya desarrolladas en el programa completo (menú, lectura de datos y ciclo principal).
- **¿Por qué es importante probar antes de integrar?**
 - Si una función falla de forma aislada, el error se encuentra en ella.
 - Si falla dentro del programa completo, el origen puede estar en la función, en la lectura de datos o en el menú, y localizarlo resulta más costoso.
 - Una función verificada puede reutilizarse con confianza.

Tipo de caso	Qué prueba	Ejemplo
Típico	Uso habitual	<code>calcular_area_rectangulo(4, 3) → 12</code>
Límite	Valores extremos del rango válido	<code>calcular_area_rectangulo(0, 5) → 0</code>
Especial	Valores atípicos o inválidos	<code>calcular_area_rectangulo(-4, 3) → -12</code> , que la función no rechaza

Pruebas con `assert`

- **Sintaxis:** Una verificación se escribe con la palabra clave `assert`, seguida de una condición y, opcionalmente, un mensaje:

```
assert condición [, "Mensaje opcional"]
```

- **Donde:**
 - **Condición:** expresión que debe ser verdadera, normalmente la comparación `llamada == resultado_esperado`.
 - **Mensaje (opcional):** texto que se muestra si la condición es falsa.
- **Si la condición se cumple**, el programa continúa sin mostrar nada.
- **Si no se cumple**, Python lanza `AssertionError` e indica la línea que falló o muestra el mensaje (en caso de que este se haya definido).

```
# Sin mensaje
assert calcular_area_rectangulo(4, 3) == 12

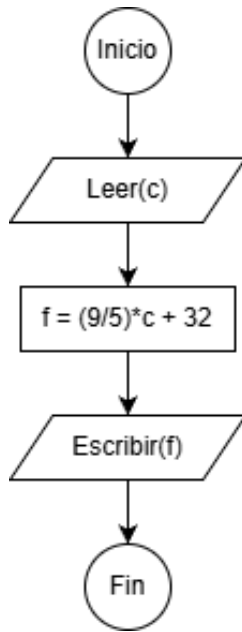
# Con mensaje
assert calcular_area_rectangulo(4, 3) == 12, "El área de 4 x 3 debe ser 12"
```

[Link simulación](#)

Salida en pantalla (Vacía – Todo salió bien)

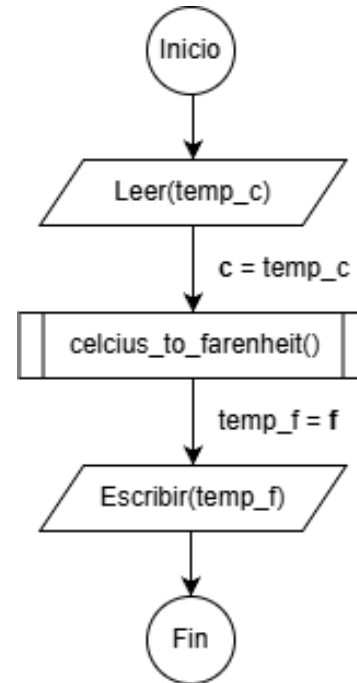
Comparación

- A continuación, se muestra la comparación entre trabajar un programa **sin módulos y con módulos**.



```
temp_c = float(input("Ingrese los °C: "))
temp_f = (9 / 5) * temp_c + 32
print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)



```
def celsius_a_fahrenheit(c):
    f = (9 / 5) * c + 32
    return f
```

```
temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)

print(f"{temp_c} °C = {temp_f} °F")
```

[Link simulación](#)

Ejemplo – Conversión de temperatura

Una estación meteorológica necesita un programa que convierta temperaturas medidas en grados Celsius (°C) a otras dos escalas: grados Fahrenheit (°F) y Kelvin (K). Al iniciar, el programa muestra un título y luego un menú con tres opciones:

1. °C a °F
2. °C a K
3. Salir

El usuario elige una opción y el programa hace lo siguiente:

- **Opción 1:** lee una temperatura en °C, la convierte a °F con la fórmula $F = (9/5)C + 32$ y muestra el resultado con dos decimales.
- **Opción 2:** lee una temperatura en °C, la convierte a Kelvin con la fórmula $K = C + 273.15$ y muestra el resultado con dos decimales.
- **Opción 3:** muestra un mensaje de despedida y termina el programa. Cualquier otro número: muestra el mensaje ERROR: Opcion no valida y vuelve a mostrar el menú.

Después de cada conversión, o de un error, el menú se muestra otra vez. El programa se repite hasta que el usuario elige la opción 3.

Ejemplo – Conversión de temperatura

Requisito: el programa debe estar organizado con funciones:

- Una función que muestre el menú.
- Una función por cada conversión, que reciba la temperatura en °C como parámetro y **devuelva** el valor convertido. Las funciones de conversión no leen ni imprimen nada: eso lo hace el programa principal.

Pruebe cada función de conversión por separado antes de integrarla al programa principal.

Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- **Entradas:**
 - La opción del menú (entero)
 - La temperatura en °C (real).
- **Salidas:**
 - La temperatura convertida con dos decimales.
 - Mensaje de error o de despedida.

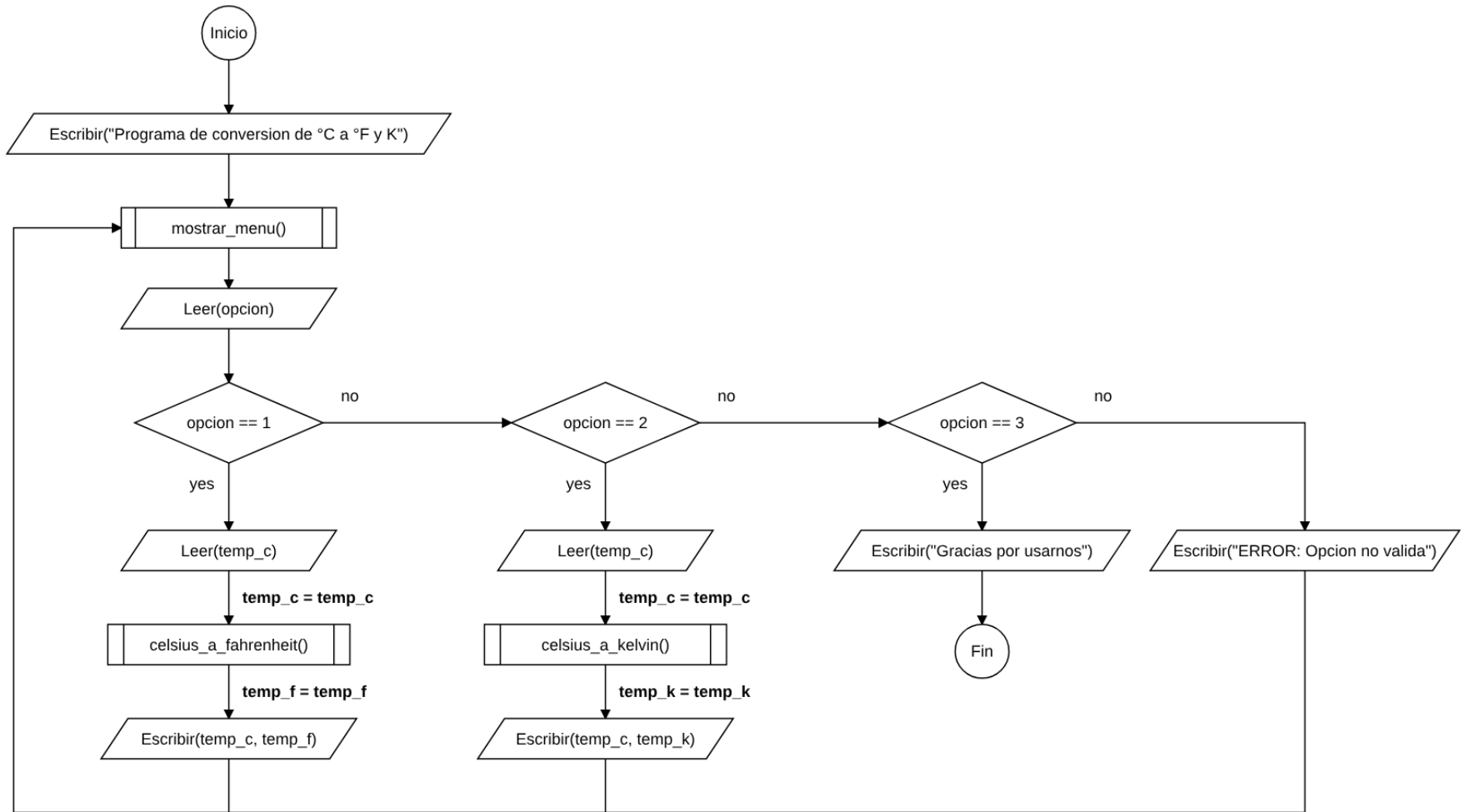
Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- Casos de prueba:

Opción	Entrada (°C)	Salida esperada	Tipo
1	100	100.0°C = 212.00°F	Típico (punto de ebullición)
1	0	0.0°C = 32.00°F	Típico (punto de congelación)
1	37	37.0°C = 98.60°F	Típico (temperatura corporal)
1	-40	-40.0°C = -40.00°F	Especial (las dos escalas coinciden)
2	25	25.0°C = 298.15K	Típico
2	-273.15	-273.15°C = 0.00K	Límite (cero absoluto)
7	–	ERROR: Opcion no valida	Especial (opción inexistente)
3	–	Gracias por usarnos	Salida del programa

Diagrama de flujo del programa



Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/defining-your-own-python-function/>
8. <https://realpython.com/python-built-in-functions/>
9. <https://realpython.com/ref/best-practices/functions/>
10. <https://makecode.microbit.org/courses/csintro>
11. <https://makecode.microbit.org/courses>
12. <https://realpython.com/python-while-loop/>
13. <https://realpython.com/ref/keywords/while/>
14. <https://cs.stanford.edu/people/nick/py/>

Eso es todo amigos

